7.6 What to do after catching an exception

Keep the models, Response, base Controller, base template, and entry boundary from 7.5. We now give both known calorie failures a useful correction path and introduce a separate optional-feature fallback. The preview does not save data or use a database.

Handle optional content with a safe fallback

Create `src/Tutorials/Class7/NoticeProvider.php`:

```
<?php
declare(strict_types=1);

namespace App\Tutorials\Class7;

use JsonException;

class NoticeProvider
{
    public function __construct(
        private string $json = '{"message":"Plan a dessert for your next campus
gathering."}',
    ) {
    }

    /** @return array{message: string, fallback: bool} */
    public function getNotice(): array
    {
        try {
            $data = json_decode($this->json, true, 512, JSON_THROW_ON_ERROR);
        } catch (JsonException $e) {
            return $this->defaultNotice();
        }

        if (!is_array($data) || !isset($data['message'])
            || !is_string($data['message']) || trim($data['message']) === '') {
            return $this->defaultNotice();
        }

        return ['message' => $data['message'], 'fallback' => false];
    }

    /** @return array{message: string, fallback: bool} */
    private function defaultNotice(): array
    {
        return [
            'message' => 'Welcome to the dessert planner.',
            'fallback' => true,
        ];
```

```
    }
}
```

The JSON string stands in for content supplied by another component. No network request is involved. JSON_THROW_ON_ERROR makes malformed JSON throw JsonException. We catch that specific failure because we can supply an acceptable default notice. Successfully decoded data still needs a shape check; wrong shape is handled with an ordinary conditional.

The returned fallback flag makes the substitution explicit. A notice is optional; the dessert itself is not. Replacing a failed calorie calculation with zero would misrepresent the result, even though zero is a valid calorie value in some other case.

Validate input before calling the model

Replace `src/Tutorials/Class7/DessertController.php`:

```
<?php
declare(strict_types=1);

namespace App\Tutorials\Class7;

class DessertController extends Controller
{
    public function preview(int $calories): Response
    {
        return $this->form([
            'name' => 'Berry crumble',
            'calories' => (string) $calories,
        ], true);
    }

    public function form(array $input, bool $submitted): Response
    {
        $nameInput = $input['name'] ?? '';
        $calorieInput = $input['calories'] ?? '';
        $name = is_string($nameInput) ? $nameInput : '';
        $calorieText = is_string($calorieInput) ? $calorieInput : '';
        $errors = [];
        $dessert = null;

        if ($submitted) {
            if (!is_string($nameInput) || trim($name) === '') {
                $errors[] = 'Enter a dessert name.';
            } elseif (strlen(trim($name)) > 80) {
                $errors[] = 'Use a dessert name of at most 80 bytes.';
            }

            $calories = false;
            if (!is_string($calorieInput)
                || preg_match('/^-?(?:0|[1-9][0-9]*)$/D', trim($calorieText)) !==
1) {
```

```
                $errors[] = 'Enter calories as a whole number, such as 500.';
            } else {
                $calories = filter_var(trim($calorieText), FILTER_VALIDATE_INT);
                if ($calories === false) {
                    $errors[] = 'That whole number is too large for this PHP
runtime.';
                }
            }

            if ($errors === []) {
                try {
                    $dessert = new Dessert(trim($name), $calories);
                } catch (NegativeCaloriesException $e) {
                    $errors[] = 'Calories must be zero or greater.';
                } catch (ExcessiveCaloriesException $e) {
                    $errors[] = 'Calories must be no more than 5000.';
                }
            }
        }

        $notice = (new NoticeProvider())->getNotice();
        $body = $this->twig->render('planner.html.twig', [
            'name' => $name,
            'calorieText' => $calorieText,
            'errors' => $errors,
            'dessert' => $dessert,
            'notice' => $notice,
        ]);
        return new Response($body, $errors === [] ? 200 : 422);
    }
}
```

Request fields may be missing or arrays. Shape checks prevent passing an array into trim or a string directly into an int parameter. We preserve the original scalar strings for redisplay while using trimmed copies for validation and construction. An array-shaped field cannot be redisplayed as text; it becomes an empty string with a correction message.

Our documented integer format allows zero, positive integers, and a leading minus sign; it rejects decimals, exponent notation, a plus sign, and leading zeros other than zero itself. filter_var then checks the runtime's integer range. Use `=== false`, not a truthiness test: zero is a valid result. The 80-byte name limit is a simple request rule; it is not a Unicode character count.

Ordinary missing or malformed form fields are handled with conditionals. The model independently enforces its domain rule and can throw when another caller violates it. The controller catches only those two known domain types and converts each to approved correction text. Notice retrieval and Twig rendering remain outside that domain try block; their unexpected failures will not be mistaken for bad calorie input.

Wire the form route and view

Replace `src/Tutorials/Class7/Application.php`:

```php
<?php
declare(strict_types=1);

namespace App\Tutorials\Class7;

class Application
{
    public function run(): Response
    {
        $action = $_GET['action'] ?? 'planner';
        if (!is_string($action)) {
            $action = 'planner';
        }

        $controller = new DessertController();
        switch ($action) {
            case 'negative':
                return $controller->preview(-1);
            case 'excessive':
                return $controller->preview(5001);
            case 'valid':
                return $controller->preview(500);
            case 'planner':
            default:
                $submitted = ($_SERVER['REQUEST_METHOD'] ?? 'GET') === 'POST';
                return $controller->form($submitted ? $_POST : [], $submitted);
        }
    }
}
```

Create `templates/tutorials/class7/planner.html.twig`:

```twig
{% extends 'base.html.twig' %}

{% block pageTitle %}Plan a dessert{% endblock %}

{% block main %}
    <aside class="notice">
        {{ notice.message }}
        {% if notice.fallback %}<small>(Default notice)</small>{% endif %}
    </aside>

    {% if errors is not empty %}
        <div class="error" role="alert">
            <p>Please correct the following:</p>
            <ul>{% for error in errors %}<li>{{ error }}</li>{% endfor %}</ul>
        </div>
    {% endif %}

    <form action="index.php" method="post">
```

```
        <label for="name">Dessert name</label>
        <input id="name" name="name" type="text" value="{{ name }}">
        <label for="calories">Calories (0–5000)</label>
        <input id="calories" name="calories" type="text" value="{{ calorieText
}}">
        <button type="submit">Preview dessert</button>
    </form>

    {% if dessert is not null %}
        <h2>Preview ready</h2>
        <p>{{ dessert.name }} contains {{ dessert.calories }} calories.</p>
        <p>This is a preview only; nothing has been saved.</p>
    {% endif %}
{% endblock %}
```

Keep the 7.5 entry script unchanged. The old preview template may remain but is no longer rendered. Missing, unknown, and array-shaped actions now choose the planner. The three demo links stay available and redisplay the same form with their preset values. The excessive demo changes from a generic 500 in 7.5 to a known correction with 422 here.

The form posts to this week's relative index.php. Text inputs intentionally allow bad values so we can see server-side validation; browser constraints are not the validation policy. Twig escapes both form values and result text. Never add the raw filter to submitted values. There is no side effect or persistent state in this lesson's POST, and no login or session feature is introduced.

Four outcomes, four decisions

Outcome	Decision	Evidence
Valid dessert	Show a complete preview	200, Preview ready, model values
Correctable input failure	Redisplay entered values with a specific correction	422, error list, no Preview ready
Optional JSON failure	Substitute an explicit default notice	Preview still usable, Default notice label
Unexpected application failure	Let it reach the entry boundary; end the request	500, generic page, no success output

We use 422 for a request whose submitted values cannot be accepted, and 500 for unexpected server failure. A correction response controls the failure; it does not make the rejected dessert exist. A user must submit corrected input in a new request. We do not automatically retry the same invalid value.

Verify recovery and termination

1. Open index.php: 200, blank form, no correction and no result. Submit Berry crumble and 500: 200 with a preview.
2. Submit -1 and then 5001: 422, the respective correction, preserved name and calorie text, no result. Submit 0 and 5000: both succeed.

3. Test blank name, whitespace-only name, 1.5, 01, an empty calorie field, and a very large integer: each gets 422. Leading/trailing spaces around an otherwise valid integer are accepted; the entered text remains in the form.
4. Use developer tools or a local request test to submit `name[]=Berry` and `calories[]=500`. Expect corrections, not TypeError. Normal browser form controls send scalar strings, so this case requires a deliberately constructed request.
5. Submit a name containing `<b>Berry</b>` and quotes. It appears as text, not markup; the input attribute remains intact.
6. Temporarily change NoticeProvider's default constructor JSON to `'{broken'`. A valid dessert still gets 200 and a labeled default notice. Restore the original JSON. Also try `'{}'`: valid JSON with missing content gets the same fallback through the conditional, not the catch.
7. To simulate an unexpected failure locally, temporarily change the render call's template name to `'missing.html.twig'`. The entry boundary returns one generic 500 document without a path or trace. Restore `'planner.html.twig'` immediately. Never create a public query parameter that deliberately triggers internal faults.

When should we not catch?

Do not catch locally when the layer cannot correct input, supply a valid fallback, release a resource as part of useful handling, or translate failure into a response it owns. Application simply propagates. The domain model throws instead of displaying HTML. A TypeError caused by a bad method call needs a code fix, even if the boundary prevents visitors from seeing its details.

If a catch has performed useful work but still cannot complete the operation, `throw $e;` propagates that same exception. A catch that does nothing except rethrow is usually unnecessary. For cleanup alone, use finally without a catch, as we will do next.

Retry is a separate policy. Consider it only for a temporary failure when repeating the operation is safe, with a strict attempt limit. Never retry an unchanged negative calorie count, a TypeError, or an operation whose earlier side effects may already have occurred. Our example needs no automatic retry.